

Task QA Runbook

Pre-rollout and post-rollout checks

1. Install

Requirements: Claude Code, Python 3, and a browser for the HTML reports. Install these five files in the QA skill folder:

```
mkdir -p ~/.claude/skills/task-quality-qa
```

- SKILL.md (Appendix A)
- eval_guide.md (Appendix B)
- POST_ROLLOUT_SKILL.md (Appendix C)
- dq_audit.py (Appendix D)
- dq_post.py (Appendix E)

Start a new Claude Code session after installation.

2. Configure

Set the path and task-layout values at the top of the supplied skill files.

3. Pre-rollout QA

1. Run the pre-rollout skill on the task directory:

```
run task-quality-preflight on <DATA_DIR>
```

2. Open report.html. For flagged findings, run the independent verification step in SKILL.md before changing the task.
3. After confirmed task fixes, rerun the affected task set.

4. Post-rollout QA

1. Prepare the rollout-results JSONL using the schema in POST_ROLLOUT_SKILL.md.
2. Run the post-rollout skill:

```
run task-quality-postrun on <ROLLOUTS>
```

3. Open post_report.html and review the evidence for repeated zero-pass criteria.

Appendix A - SKILL.md

```
---
name: task-quality-preflight
description: "Run pre-rollout QA on task prompts, grading criteria, tests, and runtime inputs; produce
JSONL findings and an HTML report."
---
# Pre-Rollout Task QA
## Configuration
DATA_DIR = <absolute path to the task set>
OUTPUT_DIR = <absolute path for QA outputs>
SKILL_DIR = <absolute path to this QA skill folder>
PROMPT = <prompt file or field>
RUBRIC = <rubric file/field and weight/points field>
TESTS = <mechanical test location, if present>
INPUTS = <files/data available to the task at runtime>
IGNORE = <solution/reference-answer or other non-runtime paths that must not be inspected>
## 1. Build one review record per task
For each task, write `OUTPUT_DIR/batch_NNN.json` as a one-element JSON array. Use a stable task order and
a zero-padded index (`000`, `001`, ...).
```json
[{"task_id": "<stable id>", "task_name": "<display name>", "task_dir": "<absolute task
directory>", "prompt_text": "<full
prompt>", "rubrics": [{"number": 1, "points": 10, "criterion": "<full grading criterion>"}], "test_code": "<test
source or empty string>", "test_sh": "<test shell source or
empty string>", "test_weights": {}, "ignore_paths": ["<absolute path not visible at runtime>"]}]]
```

Normalize each grading criterion to `{number, points, criterion}`. `number` is the criterion index;
`points` is its weight; `criterion` must contain the complete
criterion, including any sub-criteria and deterministic expected values that need to be checked against
the task inputs.
Resolve `IGNORE` into `ignore_paths` for each task so the reviewer knows exactly which paths are off
limits.
If there are no mechanical tests, use empty strings for `test_code` and `test_sh` and `{}` for
`test_weights`. Confirm `prompt_text` and `rubrics` are populated
before review.
## 2. Review each task
Use one independent reviewer/subagent per `batch_NNN.json`. Each reviewer writes
`OUTPUT_DIR/verdict_NNN.json`.
### Reviewer prompt
Read `<SKILL_DIR>/eval_guide.md` and follow its decision rules and exact output schema.
Read `<OUTPUT_DIR>/batch_NNN.json`.
1. Inspect the prompt and all inputs/resources the task makes available at runtime. Validate data through
the same interface, filtering, and scoping the task
exposes, not only by reading raw backing files. Use appropriate local tools for the input formats
present. Do not inspect any path in `ignore_paths`, including
solution/reference-answer material.
2. Apply every dimension in `eval_guide.md`. Review every grading criterion and every mechanical test.
Reproduce concrete counts, IDs, dates, totals, and
other deterministic values from the actual task data rather than guessing.
3. If there are no mechanical tests, set `problematic_tests` to `[]` and `test_quality_rationale` to
`"No tests in this task."`.
4. Write only a one-element JSON array using the exact schema in `eval_guide.md` to
`<OUTPUT_DIR>/verdict_NNN.json`. `problematic_rubrics` and
`problematic_tests` must be JSON-encoded strings; `rubric_number` must match the rubric `number`; include
`task_name`.
Ensure every expected `verdict_NNN.json` exists before continuing.
## 3. Build the report
```bash
python3 <SKILL_DIR>/dq_audit.py merge --batch-dir <OUTPUT_DIR> --output <OUTPUT_DIR>/results.jsonl --html
<OUTPUT_DIR>/report.html
```
## 4. Verify flagged findings
For each row in `results.jsonl` with `num_flags > 0`:
1. Write `OUTPUT_DIR/fp/fp_input_NNN.json` containing that row plus `"line_idx": <row index>`.
2. Have a separate reviewer read `eval_guide.md` and the corresponding `fp_input_NNN.json`, then
independently re-evaluate each flagged dimension from
scratch against the actual task data in `task_dir`. If the evidence is ambiguous, keep the finding.
```

3. Write `OUTPUT\_DIR/fp/fp\_check\_NNN.json`:

```
```json
{"line_idx":0,"task_name":"<name>","corrections":{"<field>":"<replacement
value>"},"fp_summary":[{"dimension":"<field/dimension>","verdict":"TRUE|FALSE_POSITIVE","reason":"<brief
evidence>"}]}}
```
```

Copy `results.jsonl` to `results\_v2.jsonl`. For each `fp\_check\_NNN.json`, overwrite only the fields named in `corrections` on the row identified by `line\_idx`, then

run:

```
```bash
python3 <SKILL_DIR>/dq_audit.py recompute --input <OUTPUT_DIR>/results_v2.jsonl
python3 <SKILL_DIR>/dq_audit.py report --input <OUTPUT_DIR>/results_v2.jsonl --output
<OUTPUT_DIR>/report_v2.html
```
```

Appendix B - eval_guide.md

Task QA Evaluation Guide

Evaluate one task using its prompt, grading artifacts, tests, and the environment/resources available to the task at runtime. Verify concrete claims directly from the task data and write the JSON verdict defined below.

Operating assumptions

- These tasks may intentionally use concise or open-ended prompts. The task can require the model to explore its inputs to determine the specific work to perform.

- Judge availability from the task's actual runtime view. Rubrics, tests, answer keys, and reference solutions are grading-side artifacts unless the task explicitly exposes them.

- Resources that are legitimately discoverable or obtainable in the task runtime count as available. Do not treat a grading artifact's presence in the task directory as evidence that the model can see it.

- A task may be graded by rubric criteria, mechanical tests, or both. Evaluate the two mechanisms together where the guide says to assess combined coverage.

1. Prompt clarity

`prompt\_clarity` (1-5):

- 1: the objective is contradictory or cannot be determined even after examining the task data.

- 2: the instruction is so vague that there is no discernible direction.

- 3: meaningful ambiguity remains after examining the available inputs, preventing the model from determining the correct action for a substantial part of the task.

- 4: the overall intent is clear, with only minor ambiguity.

- 5: the objective and expected action are clear.

Do not lower the score merely because the prompt omits explicit file paths or does not enumerate every subtask when the required work can reasonably be inferred from the runtime data.

`is\_agentic` (yes/no): whether completing the task requires multi-step exploration/tool use rather than a single-shot answer.

2. Grading-criterion integrity

Review every rubric criterion. For each criterion, assess all of the following:

1. ****Evaluability.**** A grader must be able to make a reliable pass/fail judgment. A criterion is not evaluable if it is too vague to distinguish pass from fail.

2. ****Accuracy.**** The criterion must agree with what the prompt and task data jointly require. It is inaccurate when it conflicts with the prompt or with the data actually available under the task's runtime scoping.

3. ****Weight direction.**** The sign of a weighted criterion must match its intent. A `sign\_error` exists when desired behavior is assigned a negative weight or undesired behavior is assigned a positive weight. This includes desired criteria phrased as avoiding, excluding, or not doing something.

Verification rules:

- Read the actual task data before judging a criterion. Reproduce embedded counts, IDs, dates, totals, and other deterministic values from the source data.

- Verify values through the same interface, filtering, permissions, or scoping the model actually encounters. Raw backing files may contain records or ranges that are not visible through the task interface.

- A value or action implied deterministically by the task data can be a valid grading target even when the prompt does not state that value verbatim.

- Deterministic answer-key criteria are valid when the expected answer can be derived from the task data.

- Do not treat intentionally absent information as a defect when the grading criteria coherently make that absence part of the challenge, such as testing inference, hallucination resistance, or graceful handling of missing data.

- Resources legitimately discoverable at runtime count as available.

- Use `overly\_specific` only when the criterion selects one arbitrary choice among multiple equally valid choices without task/data justification.

- Use `inaccurate` only for a genuine conflict with the prompt or task data, not simply because a value is not restated in the prompt.

`problematic\_rubrics` issue values: `not\_evaluable`, `inaccurate`, `sign\_error`, `overly\_specific`.

3. Mechanical-test correctness

If there are no mechanical tests, set `problematic\_tests` to `""` and `test\_quality\_rationale` to `No tests in this task.`", then evaluate coverage using the rubric criteria alone.

Otherwise inspect every test function and judge the actual assertions rather than the test name or comments.

- First determine whether the same requirement is already covered by a rubric. If so, a weak test may be only a redundant safety check; flag it only when the test itself contains a substantive defect.
- Flag incorrect expected values or endpoints.
- Flag assertions that are tautological or effectively cannot fail.
- Flag change-only comparisons, such as checking that a value changed from a seed state, when a verifiable correct target exists and should be asserted instead.
- Flag matching rules, tolerances, or regular expressions that are unjustifiably strict or fragile.
- Keyword-list matching for free-form content is not inherently defective; flag it only when the allowed list is unreasonably narrow for valid responses.

`problematic\_tests` `issue\_type` values: `incorrect\_assertion`, `wrong\_endpoint`, `brittle\_match`, `tautological`, `other`.

4. Coverage and weighting

Assess rubric criteria and mechanical tests as one combined grading pool. The scoring basis is the sum of weights for passed grading items divided by the sum of positive weights. A requirement covered by a valid test does not also need a rubric criterion, and vice versa.

`criteria\_completeness` (1-5):

- 1: major required outcomes have no rubric or test coverage.
- 3: the core outcomes are covered, but meaningful gaps remain.
- 5: all important, gradable outcomes are covered with no meaningful gaps.

Do not count as uncovered a requirement that is already verified by a deterministic answer-key criterion, or an internal execution detail that no grader can reliably observe, such as hidden reasoning or exact tool-call order.

`uncovered\_requirements`: comma-separated list or empty string.

`has\_redundancies` (yes/no): whether rubric/test items materially grade the same requirement more than once.

`score\_distribution\_reasonable` (yes/no): whether the most important requirements receive the greatest weight and all weight signs match their intended

behavior. If any rubric has a `sign\_error`, return `no`. Do not use this field to represent factual inaccuracy in a criterion; factual errors belong under grading-criterion integrity.

5. Task/input adequacy

`is\_self\_contained` (yes/no): whether the task can be completed as defined using the provided runtime data/environment plus resources legitimately discoverable at runtime. Return `no` only when the task is genuinely unsolvable, not for a minor imperfection.

`input\_adequacy` (1-5):

- 1: critical information is missing and the task cannot be completed.
- 3: most required information is available, but notable gaps remain.
- 5: all required data is available.

Before scoring input adequacy below 5, check whether the apparent absence is intentionally part of what the task is testing. Runtime-discoverable data counts as available.

`environment\_adequacy` (1-5): whether the runtime tools/services are sufficient for the task. Reserve scores of 1-2 for a genuinely missing specialized capability. Do not lower environment adequacy because a valid service/query returns no matching data; that is a data/input question rather than a missing-tool problem.

`missing\_data`: comma-separated list or empty string.

`missing\_tools`: comma-separated list or empty string. Include only genuinely missing specialized services/capabilities, not ordinary filesystem, document, spreadsheet, PDF, CSV, or image handling that is available in the runtime.

Required output

Write only a one-element JSON array to the assigned verdict file:

```
```json
[{"task_name": "<from
batch>", "prompt_clarity": 4, "is_agentic": "yes", "prompt_quality_rationale": "<brief>", "problematic_rubrics":
"[{\\"rubric_number\\": 1, \\"criterion\\": \\"...\\", \\"issue\\": \\"inaccur
ate\\", \\"rationale\\": \\"...\\"}]", "rubric_quality_rationale": "<brief>", "problematic_tests": "[]", "test_qualit
y_rationale": "<brief>", "criteria_completeness": 4, "uncovered_requi
rements": "", "has_redundancies": "no", "score_distribution_reasonable": "yes", "coverage_and_balance_rationale
": "<brief>", "is_self_contained": "yes", "input_adequa
cy": 5, "environment_adequacy": 5, "missing_data": "", "missing_tools": "", "task_adequacy_rationale": "<brief>", "
error": ""}]
```
```

``problematic_rubrics`` and ``problematic_tests`` must be strings containing JSON arrays. ``rubric_number`` must match the rubric ``number``. ``task_name`` is required.

Appendix C - POST_ROLLOUT_SKILL.md

```
---
name: task-quality-postrun
description: "Run post-rollout QA from task-level rollout results; identify repeated zero-pass criteria,
assign root causes, and produce an HTML report."
---
# Post-Rollout Task QA
## Configuration
ROLLOUTS = <path to rollout-results JSONL>
DATA_DIR = <absolute path to the corresponding task set>
OUTPUT_DIR = <absolute path for post-rollout QA outputs>
SKILL_DIR = <absolute path to this QA skill folder>
## Rollout input
`ROLLOUTS` must contain one JSON object per task/sample:
```json
{"task_id": "<stable id matching DATA_DIR>", "model": "<model/run
label>", "reward": 0.73, "per_criterion": {"1": 1, "2": 0, "3": 1}}
```

`reward` is the task-level score (0-1). `per_criterion` maps each rubric number to its criterion outcome,
where `0` means not passed and a positive value means
passed/scored. Emit one row per task/sample. Convert other grader output formats to this schema before
running the analysis.
## 1. Aggregate results
```bash
python3 <SKILL_DIR>/dq_post.py stats --rollouts <ROLLOUTS> --output-dir <OUTPUT_DIR>
```

This creates `post_summary.jsonl` plus one `rca_input_NNN.json` for each task with at least one criterion
that received a zero outcome in every reported
sample.
## 2. Review repeated zero-pass criteria
Use one independent reviewer/subagent per `rca_input_NNN.json`.
### RCA reviewer prompt
Read `<OUTPUT_DIR>/rca_input_NNN.json` and locate the matching task under `DATA_DIR`. For every entry in
`zero_pass_criteria`, read the corresponding
grading criterion and inspect the actual task inputs plus any grader/output evidence available for that
rollout.
Assign exactly one `primary_issue` based on direct evidence:
- `GENUINE_FAILURE`: the task is solvable and the grading criterion is valid; the rollout outputs did not
satisfy it.
- `RUBRIC_INACCURATE`: the criterion contains an expected value, ID, count, fact, or requirement that
conflicts with the actual task data.
- `RUBRIC_DESIGN`: the criterion is not reliably evaluable, is arbitrarily over-specific, or has the
wrong weight direction.
- `DATA_ISSUE`: data required by the criterion is missing, incorrect, or contradictory in the runtime
inputs.
- `GRADER_FALSE_POSITIVE`: a correct/valid output can be rejected by the grading implementation.
- `GRADER_TRUNCATION`: the available rollout evidence shows the output was cut off before the grader
could evaluate the relevant content.
Verify against the real task data and available grading evidence; do not infer the root cause from
failure frequency alone.
Write `<OUTPUT_DIR>/rca_verdict_NNN.json`:
```json
{"task_id": "<id>", "criteria_rca": [{"criterion_key": "<rubric number>", "primary_issue": "<label
above>", "evidence": "<brief
evidence>"}], "task_level_assessment": "<one-line summary>"}
```
## 3. Build the report
`python3 <SKILL_DIR>/dq_post.py report --output-dir <OUTPUT_DIR> --output <OUTPUT_DIR>/post_report.html`
```

Appendix D - dq_audit.py

Pre-rollout merge/report helper. Python 3 standard library only. Used by SKILL.md steps 3 and 4 (merge, recompute, report).

```
#!/usr/bin/env python3
"""Portable RL data quality audit helper – stdlib only, no external services.

Subcommands:
    merge --batch-dir DIR --output results.jsonl --html report.html
        Join verdict_*.json to batch_*.json, compute weights + flags, write JSONL + HTML.
    report --input results.jsonl --output report.html
        Re-render the HTML report from an existing results JSONL.
    recompute --input results.jsonl [--output out.jsonl]
        Recompute weight aggregates + flags in place (use after editing verdicts).

The batch files (batch_NNN.json) are written by Claude Code following the skill
(one JSON array with one task object). This helper never talks to any network
service – reports are local HTML files you open in a browser.
"""
from __future__ import annotations
import argparse, glob, html as html_lib, json, os, re
from pathlib import Path

# ---- fields we track per task ----
STR_FIELDS = ["task_id", "task_name", "source", "is_agentic", "prompt_quality_rationale",
              "problematic_rubrics", "rubric_quality_rationale", "problematic_tests",
              "test_quality_rationale",
              "uncovered_requirements", "has_redundancies", "score_distribution_reasonable",
              "coverage_and_balance_rationale", "is_self_contained", "missing_data", "missing_tools",
              "task_adequacy_rationale", "task_dir", "prompt_text", "rubrics_json", "test_weights_json",
              "flagged_dimensions", "error"]
INT_FIELDS = ["prompt_clarity", "criteria_completeness", "input_adequacy", "environment_adequacy",
              "num_problematic_rubrics", "num_total_rubrics", "num_problematic_tests", "num_total_tests",
              "num_flags"]
FLOAT_FIELDS = ["problematic_weight", "total_positive_weight", "total_absolute_weight",
                "rubric_problematic_weight", "rubric_absolute_weight"]

def _blank_row():
    r = {k: "" for k in STR_FIELDS}
    r.update({k: 0 for k in INT_FIELDS})
    r.update({k: 0.0 for k in FLOAT_FIELDS})
    return r

def _parse_rubric_scores(rubrics_json):
    scores = {}
    try:
        for i, rb in enumerate(json.loads(rubrics_json), start=1):
            rn = str(rb.get("number", rb.get("rubric_number", "")))
            if rn.startswith("R"):
                rn = rn[1:]
            if not rn:
                rn = str(i)
            scores[rn] = float(rb.get("points", rb.get("score", rb.get("weight", 0))) or 0)
    except (json.JSONDecodeError, TypeError, ValueError):
        pass
    return scores

def _parse_test_weights(tw_json):
    try:
        return {k: float(v) for k, v in json.loads(tw_json).items()}
    except (json.JSONDecodeError, TypeError, ValueError):
        return {}
```

```

def _compute_weights(r):
    rubric_scores = _parse_rubric_scores(r["rubrics_json"]) if r["rubrics_json"] else {}
    tw = _parse_test_weights(r["test_weights_json"]) if r["test_weights_json"] else {}
    test_weights = list(tw.values()) if tw else ([3.0] * r["num_total_tests"] if r["num_total_tests"]
else [])
    r["rubric_absolute_weight"] = sum(abs(w) for w in rubric_scores.values())
    r["total_positive_weight"] = sum(w for w in rubric_scores.values() if w > 0) + sum(w for w in
test_weights if w > 0)
    r["total_absolute_weight"] = r["rubric_absolute_weight"] + sum(abs(w) for w in test_weights)
    prob = 0.0
    if r["problematic_rubrics"]:
        try:
            for p in json.loads(r["problematic_rubrics"]):
                prob += abs(rubric_scores.get(str(p.get("rubric_number", "")), 0))
            except (json.JSONDecodeError, TypeError, ValueError):
                pass
    r["rubric_problematic_weight"] = prob
    tprob = 0.0
    if r["problematic_tests"]:
        try:
            for p in json.loads(r["problematic_tests"]):
                tprob += abs(tw.get(str(p.get("test_name", "")), 3.0))
            except (json.JSONDecodeError, TypeError, ValueError):
                pass
    r["problematic_weight"] = prob + tprob

def _compute_flags(r):
    flags = []
    if r["prompt_clarity"] and r["prompt_clarity"] <= 3:
        flags.append("prompt_clarity")
    if r["total_absolute_weight"] > 0 and r["problematic_weight"] / r["total_absolute_weight"] >= 0.10:
        flags.append("problematic_rubrics_tests")
    if r["criteria_completeness"] and r["criteria_completeness"] <= 3:
        flags.append("criteria_completeness")
    if str(r["score_distribution_reasonable"]).lower() == "no":
        flags.append("score_distribution")
    if r["input_adequacy"] and r["input_adequacy"] <= 3:
        flags.append("input_adequacy")
    if r["environment_adequacy"] and r["environment_adequacy"] <= 3:
        flags.append("environment_adequacy")
    if str(r["is_self_contained"]).lower() == "no":
        flags.append("not_self_contained")
    r["num_flags"] = len(flags)
    r["flagged_dimensions"] = ", ".join(flags)

# ---- merge ----
def _norm_list_field(v):
    if isinstance(v, list):
        return json.dumps(v) if v else ""
    return v or ""

def merge(batch_dir, out_jsonl, out_html):
    verdicts = sorted(glob.glob(os.path.join(batch_dir, "verdict_*.json")))
    if not verdicts:
        raise SystemExit(f"No verdict_*.json in {batch_dir}")
    rows = []
    eval_fields = ["prompt_clarity", "is_agentic", "prompt_quality_rationale", "problematic_rubrics",
"rubric_quality_rationale", "criteria_completeness", "uncovered_requirements",
"has_redundancies",
"score_distribution_reasonable", "coverage_and_balance_rationale",
"is_self_contained",
"input_adequacy", "environment_adequacy", "missing_data", "missing_tools",
"task_adequacy_rationale",
"problematic_tests", "test_quality_rationale", "error"]
    for vf in verdicts:

```

```

idx = os.path.basename(vf).split("verdict_")[1].split(".")[0]
bp = os.path.join(batch_dir, f"batch_{idx}.json")
bmeta = {}
if os.path.isfile(bp):
    bt = json.load(open(bp))
    bt = bt if isinstance(bt, list) else [bt]
    for bd in bt:
        rubrics = bd.get("rubrics", [])
        tc = bd.get("test_code", "") or ""
        bmeta[bd.get("task_name", "")] = {
            "task_id": bd.get("task_id", ""), "task_name": bd.get("task_name", ""),
            "source": bd.get("source", ""), "task_dir": bd.get("task_dir", ""),
            "prompt_text": bd.get("prompt_text", ""),
            "rubrics_json": json.dumps(rubrics) if isinstance(rubrics, list) else (rubrics or
""),

            "num_total_rubrics": len(rubrics) if isinstance(rubrics, list) else 0,
            "num_total_tests": len(re.findall(r"def test_", tc)) if tc else 0,
            "test_weights_json": json.dumps(bd.get("test_weights", {})),
        }
raw = json.load(open(vf))
for v in (raw if isinstance(raw, list) else [raw]):
    if not isinstance(v, dict):
        continue
    r = _blank_row()
    name = v.get("task_name", "")
    meta = bmeta.get(name) or (next(iter(bmeta.values())) if len(bmeta) == 1 else {})
    r.update({k: meta.get(k, r[k]) for k in meta})
    for k in eval_fields:
        if k in v and v[k] is not None:
            r[k] = v[k]
    r["problematic_rubrics"] = _norm_list_field(r["problematic_rubrics"])
    r["problematic_tests"] = _norm_list_field(r["problematic_tests"])
    for lf, cf in [("problematic_rubrics", "num_problematic_rubrics"),
        ("problematic_tests", "num_problematic_tests")]:
        r[cf] = len(json.loads(r[lf])) if isinstance(r[lf], str) and r[lf].startswith("[") else 0
    for k in INT_FIELDS:
        try:
            r[k] = int(r[k]) if r[k] not in ("", None) else 0
        except (TypeError, ValueError):
            r[k] = 0
    _compute_weights(r)
    _compute_flags(r)
    rows.append(r)
_write_jsonl(rows, out_jsonl)
generate_html(rows, out_html)
ok = [r for r in rows if not r["error"]]
n = max(len(ok), 1)
flagged = sum(1 for r in ok if r["num_flags"] > 0)
print(f"Merged {len(rows)} results, {len(rows)-len(ok)} errors")
print(f"Flagged: {flagged}/{len(rows)}")
print(f"Avg clarity={sum(r['prompt_clarity'] for r in ok)/n:.1f} "
    f"completeness={sum(r['criteria_completeness'] for r in ok)/n:.1f} "
    f"input_adeq={sum(r['input_adequacy'] for r in ok)/n:.1f} "
    f"env_adeq={sum(r['environment_adequacy'] for r in ok)/n:.1f}")
print(f"Problematic rubrics={sum(r['num_problematic_rubrics'] for r in ok)}")
print(f"JSONL: {out_jsonl}\nHTML: {out_html}")

def _write_jsonl(rows, path):
    with open(path, "w") as f:
        for r in rows:
            f.write(json.dumps(r) + "\n")

def _load_jsonl(path):
    rows = []
    for line in open(path):
        if not line.strip():

```

```

        continue
    d = _blank_row()
    raw = json.loads(line)
    d.update({k: raw.get(k, d[k]) for k in d})
    for k in INT_FIELDS:
        try:
            d[k] = int(d[k]) if d[k] not in ("", None) else 0
        except (TypeError, ValueError):
            d[k] = 0
    rows.append(d)
return rows

# ---- HTML ----
_STYLE = """
:root{--bg:#f0f2f5;--card:#fff;--border:#e4e6eb;--text:#1c1e21;--muted:#65676b;--accent:#1877f2;--green:#31a24c;--red:#e4405f}
*{box-sizing:border-box}body{font-family:-apple-system,BlinkMacSystemFont,'Segoe
UI',Roboto,Arial,sans-serif;margin:0;padding:24px;background:var(--bg);color:var(--text);line-height:1.5}
.wrap{max-width:1300px;margin:0 auto}h1{font-size:22px;margin:0 0
4px}.sub{color:var(--muted);font-size:13px;margin-bottom:16px}
.card{background:var(--card);border-radius:8px;box-shadow:0 1px 2px
rgba(0,0,0,.1);margin-bottom:16px;padding:16px 20px}
.tiles{display:grid;grid-template-columns:repeat(auto-fit,minmax(140px,1fr));gap:12px}
.tile{background:var(--bg);padding:14px;border-radius:8px;text-align:center}.tile
.n{font-size:22px;font-weight:700}.tile
.l{color:var(--muted);font-size:11px;text-transform:uppercase;letter-spacing:.5px;margin-top:2px}
table{border-collapse:collapse;width:100%;font-size:13px}thead th{background:#f8f9fa;border-bottom:2px
solid var(--border);padding:9px
11px;text-align:left;font-size:11px;text-transform:uppercase;color:var(--muted)}
tbody td{padding:9px 11px;border-bottom:1px solid
var(--border);vertical-align:middle}tr[data-i]{cursor:pointer}tr[data-i]:hover{background:#f5f7fa}tr.flag
{background:#fff5f5}tr.flag:hover{background:#fee}
.modal{display:none;position:fixed;inset:0;background:rgba(0,0,0,.45);z-index:100;justify-content:center;
align-items:flex-start;padding:40px 20px;overflow-y:auto}.modal.open{display:flex}
.box{background:#fff;border-radius:12px;max-width:900px;width:100%;padding:0 0
16px}.mh{display:flex;justify-content:space-between;padding:16px 22px;border-bottom:1px solid
var(--border)}.mh
h2{margin:0;font-size:16px}.x{cursor:pointer;font-size:22px;color:var(--muted);border:none;background:non
e}
.mc{padding:16px
22px;max-height:75vh;overflow-y:auto}.sec{margin-bottom:12px}.st{font-size:12px;font-weight:700;text-tran
sform:uppercase;color:var(--accent);margin-bottom:5px}
.rat{background:#fafbfc;border:1px solid var(--border);border-radius:6px;padding:9px
11px;font-size:12px;color:#444;white-space:pre-wrap}pre{white-space:pre-wrap;background:#fafbfc;border:1p
x solid var(--border);border-radius:6px;padding:10px;font-size:12px;max-height:300px;overflow:auto}
.issue{border:1px solid var(--border);border-radius:6px;margin-bottom:7px}.ih{padding:7px
11px;background:#fafbfc;border-bottom:1px solid
var(--border);font-size:12px}.tag{background:#fce8e6;color:#c5211f;border-radius:4px;padding:1px
7px;font-size:10px;font-weight:600;text-transform:uppercase;margin-right:7px}.ir{padding:7px
11px;font-size:12px;color:#555}
.f{display:flex;gap:8px;margin-bottom:12px}.fb{padding:6px 14px;border-radius:16px;border:1px solid
var(--border);background:#fff;font-size:12px;cursor:pointer}.fb.on{background:var(--accent);color:#fff}
"""

def _color(s):
    return "#27ae60" if s >= 4 else ("#f39c12" if s >= 3 else "#e74c3c")

def _cards(pr_json, kind):
    out = []
    try:
        items = json.loads(pr_json) if pr_json else []
    except json.JSONDecodeError:
        items = []
    for p in items:
        tag = html_lib.escape(str(p.get("issue", p.get("issue_type", ""))))

```

```

        ident = f"R{p.get('rubric_number','?')}" if kind == "rubric" else
html_lib.escape(str(p.get("test_name", "?")))
        crit = html_lib.escape(str(p.get("criterion", "")))
        rat = html_lib.escape(str(p.get("rationale", "")))
        out.append(f'<div class="issue"><div class="ih"><span class="tag">{tag}</span><b>{ident}</b>
{crit}</div>'
                    f'<div class="ir">{rat}</div></div>')
    return "".join(out)

def generate_html(rows, out_html):
    total = len(rows)
    ok = [r for r in rows if not r["error"]]
    n = max(len(ok), 1)
    flagged = sum(1 for r in ok if r["num_flags"] > 0)
    avg_c = sum(r["prompt_clarity"] for r in ok) / n
    avg_k = sum(r["criteria_completeness"] for r in ok) / n
    tpr = sum(r["num_problematic_rubrics"] for r in ok)
    p = ['<!doctype html><html><head><meta charset="utf-8"><title>RL Data Quality Audit</title>',
        f"<style>{_STYLE}</style></head><body><div class='wrap'>"]
    p.append(f"<div class='card'><h1>RL Data Quality Audit</h1><div class='sub'>{flagged} flagged /
{total} tasks "
            f"&bull; click a row for details</div>"
            f"<div class='tiles'>"
            f"<div class='tile'><div class='n'>{total}</div><div class='l'>Tasks</div></div>"
            f"<div class='tile'><div class='n' style='color:{{'#e4405f' if flagged else
'#31a24c'}}>{flagged}</div>"
            f"<div class='l'>Flagged</div></div>"
            f"<div class='tile'><div class='n' style='color:{{_color(avg_c)}}>{avg_c:.1f}</div><div
class='l'>Avg Clarity</div></div>"
            f"<div class='tile'><div class='n' style='color:{{'#e4405f' if tpr else
'#31a24c'}}>{tpr}</div>"
            f"<div class='l'>Prob. Rubrics</div></div>"
            f"<div class='tile'><div class='n' style='color:{{_color(avg_k)}}>{avg_k:.1f}</div><div
class='l'>Avg Completeness</div></div>"
            f"</div></div>")
    p.append(f"<div class='card'><div class='f'><button class='fb' data-f='all'>All ({total})</button>"
            f"<button class='fb' data-f='flag'>Flagged ({flagged})</button>"
            f"<button class='fb' data-f='clean'>Clean ({total-flagged})</button></div>"
            "<table><thead><tr><th>Task</th><th>Source</th><th>Clarity</th><th>Prob. Rubrics</th>"
            "<th>Completeness</th><th>Score"
            "Dist</th><th>Self-Contained</th><th>Input</th><th>Env</th></tr></thead><tbody>")
    details = []
    for i, r in enumerate(rows):
        fl = r["num_flags"] > 0
        p.append(f"<tr data-i='{i}' class='{{'flag' if fl else ''}}>"
                f"<td><b>{html_lib.escape(r['task_name'])}</b></td><td>{html_lib.escape(r['source'])}</td>"
                f"<td style='color:{{_color(r['prompt_clarity'])}}><b>{r['prompt_clarity']}</b></td>"
                f"<td style='color:{{'#e4405f' if r['num_problematic_rubrics'] else '#31a24c'}}>"
                f"{r['num_problematic_rubrics']}/{r['num_total_rubrics']}</td>"
                f"<td"
                f"style='color:{{_color(r['criteria_completeness'])}}><b>{r['criteria_completeness']}</b></td>"
                f"<td>{r['score_distribution_reasonable']}</td><td>{r['is_self_contained']}</td>"
                f"<td style='color:{{_color(r['input_adequacy'])}}>{r['input_adequacy']}</td>"
                f"<td"
                f"style='color:{{_color(r['environment_adequacy'])}}>{r['environment_adequacy']}</td></tr>")
        d = [f"<div class='sec'><div
class='st'>Prompt</div><pre>{html_lib.escape(r['prompt_text'][:6000])}</pre></div>",
            f"<div class='sec'><div class='st'>Prompt Quality</div><div class='rat'>"
            f"{html_lib.escape(r['prompt_quality_rationale'])}</div></div>",
            "<div class='sec'><div class='st'>Rubric Accuracy</div>" + _cards(r["problematic_rubrics"],
"rubric")
            + f"<div class='rat'>{html_lib.escape(r['rubric_quality_rationale'])}</div></div>",
            "<div class='sec'><div class='st'>Test Correctness</div>" + _cards(r["problematic_tests"],
"test")
            + f"<div class='rat'>{html_lib.escape(r['test_quality_rationale'])}</div></div>",
            f"<div class='sec'><div class='st'>Coverage & Balance</div><div class='rat'>"]

```

```

        f"{html_lib.escape(r['coverage_and_balance_rationale'])}</div></div>",
        f"<div class='sec'><div class='st'>Task Adequacy</div><div class='rat'>"
        f"{html_lib.escape(r['task_adequacy_rationale'])}</div></div>"]
    details.append(f"<div id='d{i}' style='display:none'>{''.join(d)}</div>")
    p.append("</tbody></table></div>")
    p.append("<div class='modal' id='m'><div class='box'><div class='mh'><h2 id='mt'></h2>"
            "    <button class='x'"
onclick="\document.getElementById('m').classList.remove('open')\">&times;</button></div>"
            "    <div class='mc' id='mb'></div></div></div>")
    p.append("<div style='display:none'>" + "".join(details) + "</div>")
    p.append("""<script>
document.querySelectorAll('tr[data-i]').forEach(function(t){t.onclick=function(){var
i=this.dataset.i;document.getElementById('mt').textContent=this.cells[0].innerText;document.getElementById('mb').innerHTML=document.getElementById('d'+i).innerHTML;document.getElementById('m').classList.add('open')}});
document.getElementById('m').onclick=function(e){if(e.target===this)this.classList.remove('open');
document.querySelectorAll('.fb').forEach(function(b){b.onclick=function(){document.querySelectorAll('.fb').forEach(x=>x.classList.remove('on'));this.classList.add('on');var
f=this.dataset.f;document.querySelectorAll('tr[data-i]').forEach(function(r){r.style.display=(f==='all' || (f==='flag')===r.classList.contains('flag'))?'':'none'}})});
</script></div></body></html>""")
    Path(out_html).write_text("".join(p))

def recompute(inp, out):
    rows = _load_jsonl(inp)
    for r in rows:
        _compute_weights(r)
        _compute_flags(r)
    _write_jsonl(rows, out or inp)
    print(f"Recomputed {len(rows)} rows -> {out or inp}")

def main():
    ap = argparse.ArgumentParser(description="Portable RL data quality audit helper (stdlib only)")
    sub = ap.add_subparsers(dest="cmd", required=True)
    m = sub.add_parser("merge")
    m.add_argument("--batch-dir", required=True)
    m.add_argument("--output", required=True)
    m.add_argument("--html", required=True)
    r = sub.add_parser("report")
    r.add_argument("--input", required=True)
    r.add_argument("--output", required=True)
    rc = sub.add_parser("recompute")
    rc.add_argument("--input", required=True)
    rc.add_argument("--output", default=None)
    a = ap.parse_args()
    if a.cmd == "merge":
        merge(a.batch_dir, a.output, a.html)
    elif a.cmd == "report":
        generate_html(_load_jsonl(a.input), a.output)
        print(f"HTML: {a.output}")
    elif a.cmd == "recompute":
        recompute(a.input, a.output)

if __name__ == "__main__":
    main()

```

Appendix E - dq_post.py

Post-rollout reward/RCA helper. Python 3 standard library only. Used by POST_ROLLOUT_SKILL.md steps 1 and 3 (stats, report).

```
#!/usr/bin/env python3
"""Portable POST-rollout quality helper – stdlib only, no external services.

Consumes YOUR rollout results (a local JSONL – one row per (task, sample)) and produces
per-task reward stats, cross-model zero-pass criteria, RCA dispatch inputs, and a local HTML report.

Rollout input row schema (one JSON object per line):
{
  "task_id": "<stable id>",          # required
  "model": "<model label>",          # required (>=2 distinct models recommended)
  "reward": 0.73,                    # required, task-level reward for this sample (0..1)
  "per_criterion": {"1": 1, "2": 0, ...} # required: rubric-number -> pass(1)/fail(0) or score
}

Subcommands:
stats --rollouts roll.jsonl --output-dir OUT
    Aggregate per task; write OUT/post_summary.jsonl and OUT/rca_input_NNN.json for tasks with
    >=1 cross-model zero-pass criterion; print the reward distribution.
report --output-dir OUT --output OUT/post_report.html
    Render reward buckets + zero-pass table, folding in OUT/rca_verdict_*.json if present.
"""

from __future__ import annotations
import argparse, glob, json, os, statistics as st, html as html_lib
from collections import defaultdict
from pathlib import Path

def _load_rollouts(path):
    by = defaultdict(list)
    for line in open(path):
        if not line.strip():
            continue
        r = json.loads(line)
        by[r["task_id"]].append(r)
    return by

def stats(rollouts, out_dir):
    os.makedirs(out_dir, exist_ok=True)
    by = _load_rollouts(rollouts)
    summ = []
    n_rca = 0
    for i, (tid, samples) in enumerate(sorted(by.items())):
        rewards = [float(s.get("reward", 0)) for s in samples]
        models = sorted({s.get("model", "?") for s in samples})
        # cross-model zero-pass: a criterion scored 0 by EVERY sample that reports it
        crit_max = defaultdict(float)
        seen = set()
        for s in samples:
            for k, v in (s.get("per_criterion") or {}).items():
                seen.add(k)
                crit_max[k] = max(crit_max[k], float(v))
        zero_pass = sorted(k for k in seen if crit_max[k] == 0.0)
        row = {"task_id": tid, "models": models, "n_samples": len(samples),
              "avg_reward": round(st.mean(rewards), 4), "min_reward": round(min(rewards), 4),
              "max_reward": round(max(rewards), 4), "n_criteria": len(seen),
              "n_zero_pass": len(zero_pass), "zero_pass_criteria": zero_pass}
        summ.append(row)
        if zero_pass:
            idx = f"{i:03d}"
            Path(f"{out_dir}/rca_input_{idx}.json").write_text(json.dumps(
                {"line_idx": i, "task_id": tid, "zero_pass_criteria": zero_pass,
                 "avg_reward": row["avg_reward"], "min_reward": row["min_reward"]}, indent=2))
```

```

        n_rca += 1
    with open(f"{out_dir}/post_summary.jsonl", "w") as f:
        for r in summ:
            f.write(json.dumps(r) + "\n")
    n = max(len(summ), 1)
    avgs = [r["avg_reward"] for r in summ]
    print(f"tasks={len(summ)} models={sorted({m for r in summ for m in r['models']})}")
    print(f"avg_reward mean={st.mean(avgs):.3f} median={st.median(avgs):.3f}")
    print(f"tasks avg<0.5: {sum(a<0.5 for a in avgs)} avg==0: {sum(a==0 for a in avgs)} "
          f"min<0.8: {sum(r['min_reward']<0.8 for r in summ)}")
    tot_c = sum(r["n_criteria"] for r in summ)
    zp = sum(r["n_zero_pass"] for r in summ)
    print(f"cross-model zero-pass criteria: {zp}/{tot_c} tasks w/ >=1: {n_rca}")
    print(f"wrote {out_dir}/post_summary.jsonl and {n_rca} rca_input_*.json")

```

```

_STYLE =
("body{font-family:-apple-system,Arial,sans-serif;margin:0;padding:24px;background:#f0f2f5;color:#1c1e21}
"
    ".wrap{max-width:1100px;margin:0 auto}h1{font-size:22px;margin:0 0 4px}"
    ".sub{color:#65676b;font-size:13px;margin-bottom:16px}"
    ".card{background:#fff;border-radius:8px;box-shadow:0 1px 2px rgba(0,0,0,.1);padding:16px
20px;margin-bottom:16px}"
    "h2{font-size:15px;margin:0 0 12px}table{border-collapse:collapse;width:100%;font-size:13px}"
    "th{background:#f8f9fa;text-align:left;padding:9px 11px;border-bottom:2px solid
#e4e6eb;font-size:11px;"
    "text-transform:uppercase;color:#65676b}td{padding:8px 11px;border-bottom:1px solid #eef0f2}"

".num{text-align:right;font-variant-numeric:tabular-nums}.bar{height:20px;border-radius:4px;display:inlin
e-block}")

```

```

def report(out_dir, out_html):
    summ = [json.loads(l) for l in open(f"{out_dir}/post_summary.jsonl") if l.strip()]
    verdicts = {}
    for vf in glob.glob(f"{out_dir}/rca_verdict_*.json"):
        v = json.load(open(vf))
        verdicts[v.get("task_id", "")] = v
    avgs = [r["avg_reward"] for r in summ]
    buckets = [
        ("avg == 0 (broken/unsolvable)", sum(a == 0 for a in avgs), "#e74c3c"),
        ("0 < avg < 0.5 (hard or broken)", sum(0 < a < 0.5 for a in avgs), "#e67e22"),
        ("0.5 <= avg < 0.8 (sweet spot)", sum(0.5 <= a < 0.8 for a in avgs), "#27ae60"),
        ("0.8 <= avg < 1.0 (easy-ish)", sum(0.8 <= a < 1.0 for a in avgs), "#2ecc71"),
        ("avg == 1.0 (saturated)", sum(a == 1.0 for a in avgs), "#f39c12")]
    rc = defaultdict(int)
    for v in verdicts.values():
        for c in v.get("criteria_rca", []):
            rc[c.get("primary_issue", "UNKNOWN")] += 1
    p = [f"<!doctype html><html><head><meta charset='utf-8'><title>Post-Rollout Quality</title>"
        f"<style>{_STYLE}</style></head><body><div class='wrap'>"]
    n = max(len(summ), 1)
    p.append(f"<div class='card'><h1>Post-Rollout Quality Report</h1><div class='sub'>{len(summ)} tasks
&bull; "
        f"reward + cross-model zero-pass</div>"
        f"<table><tr><th>Metric</th><th>Value</th></tr>"
        f"<tr><td>Tasks</td><td class='num'>{len(summ)}</td></tr>"
        f"<tr><td>Avg reward (mean / median)</td><td class='num'>{st.mean(avgs):.3f} /
{st.median(avgs):.3f}</td></tr>"
        f"<tr><td>Tasks min_reward &lt; 0.8</td><td class='num'>{sum(r['min_reward']<0.8 for r in
summ)}</td></tr>"
        f"<tr><td>Cross-model zero-pass criteria</td><td class='num'>{sum(r['n_zero_pass'] for r in
summ)} / "
        f"{sum(r['n_criteria'] for r in summ)}</td></tr>"
        f"<tr><td>Tasks with &ge;1 zero-pass</td><td class='num'>{sum(1 for r in summ if
r['n_zero_pass']>0)}</td></tr>"
        f"</table></div>")
    mx = max((c for _, c, _ in buckets), default=1) or 1
    p.append("<div class='card'><h2>Reward distribution (avg across models)</h2>")

```

```

        "<table><tr><th>Bucket</th><th>Tasks</th><th></th></tr>")
    for lab, c, col in buckets:
        p.append(f"<tr><td>{lab}</td><td class='num'>{c}</td><td><span class='bar' "
            f"style='width:{int(500*c/mx)}px;background:{col}'></span></td></tr>")
    p.append("</table></div>")
    if rc:
        p.append("<div class='card'><h2>RCA root-cause breakdown (zero-pass criteria)</h2>"
            "<table><tr><th>Primary issue</th><th>Count</th></tr>")
        for k, c in sorted(rc.items(), key=lambda x: -x[1]):
            p.append(f"<tr><td>{html_lib.escape(k)}</td><td class='num'>{c}</td></tr>")
        p.append("</table></div>")
    p.append("<div class='card'><h2>Tasks with zero-pass"
        criteria</h2><table><tr><th>task_id</th><th>avg</th>"
        "<th>min</th><th>zero-pass</th><th>RCA</th></tr>")
    for r in sorted(summ, key=lambda x: (-x["n_zero_pass"], x["avg_reward"])):
        if r["n_zero_pass"] == 0:
            continue
        v = verdicts.get(r["task_id"], {})
        issues = ", ".join(c.get("primary_issue", "") for c in v.get("criteria_rca", [])) if v else ""
        p.append(f"<tr><td>{html_lib.escape(r['task_id'])}</td><td"
            class='num'>{r['avg_reward']:.2f}</td>"
            f"<td class='num'>{r['min_reward']:.2f}</td>"
            f"<td"
            class='num'>{r['n_zero_pass']}</td><td>{html_lib.escape(issues)}</td></tr>")
        p.append("</table></div></div></body></html>")
    Path(out_html).write_text("".join(p))
    print(f"HTML: {out_html}")

def main():
    ap = argparse.ArgumentParser(description="Portable post-rollout quality helper (stdlib only)")
    sub = ap.add_subparsers(dest="cmd", required=True)
    s = sub.add_parser("stats")
    s.add_argument("--rollouts", required=True)
    s.add_argument("--output-dir", required=True)
    r = sub.add_parser("report")
    r.add_argument("--output-dir", required=True)
    r.add_argument("--output", required=True)
    a = ap.parse_args()
    if a.cmd == "stats":
        stats(a.rollouts, a.output_dir)
    else:
        report(a.output_dir, a.output)

if __name__ == "__main__":
    main()

```